

COURS NLP – TRAITEMENT AUTOMATIQUE DE MANUSCRITS ANCIENS

- **Module NLP** • MD5 Volet 2 • Master Data/IA • 2026

Ces supports accompagnent les 3 jours du module NLP. Ils supposent la lecture du support de cours du module Computer Vision, dont ils constituent le prolongement direct. Le lecteur est censé avoir produit, ou étudié, un pipeline HTR fonctionnel avant d'aborder ces chapitres.

PLAN GENERAL DE LA FORMATION

Jour	Matin (09h00-12h30)	Après-midi (13h30-17h00)
J1	Introduction au TAL, Ingestion du data contract HTR (validation, EDA, split) + normalisation par règles	Correction guidée par confiance + évaluation CER
J2	Fine-tuning T5-LoRA pour la normalisation des cas résiduels	Préparation des données NER (schéma BIO, annotation manuelle rapide, alignement labels)
J3	Fine-tuning CamemBERT-LoRA (NER) + évaluation sequeval	Annotation POS avec Stanza + extraction de relations par règles + graphe NetworkX + export TEI

JOUR 1 – INGESTION DU CORPUS & NORMALISATION PAR RÈGLES

OBJECTIFS INTERMÉDIAIRES

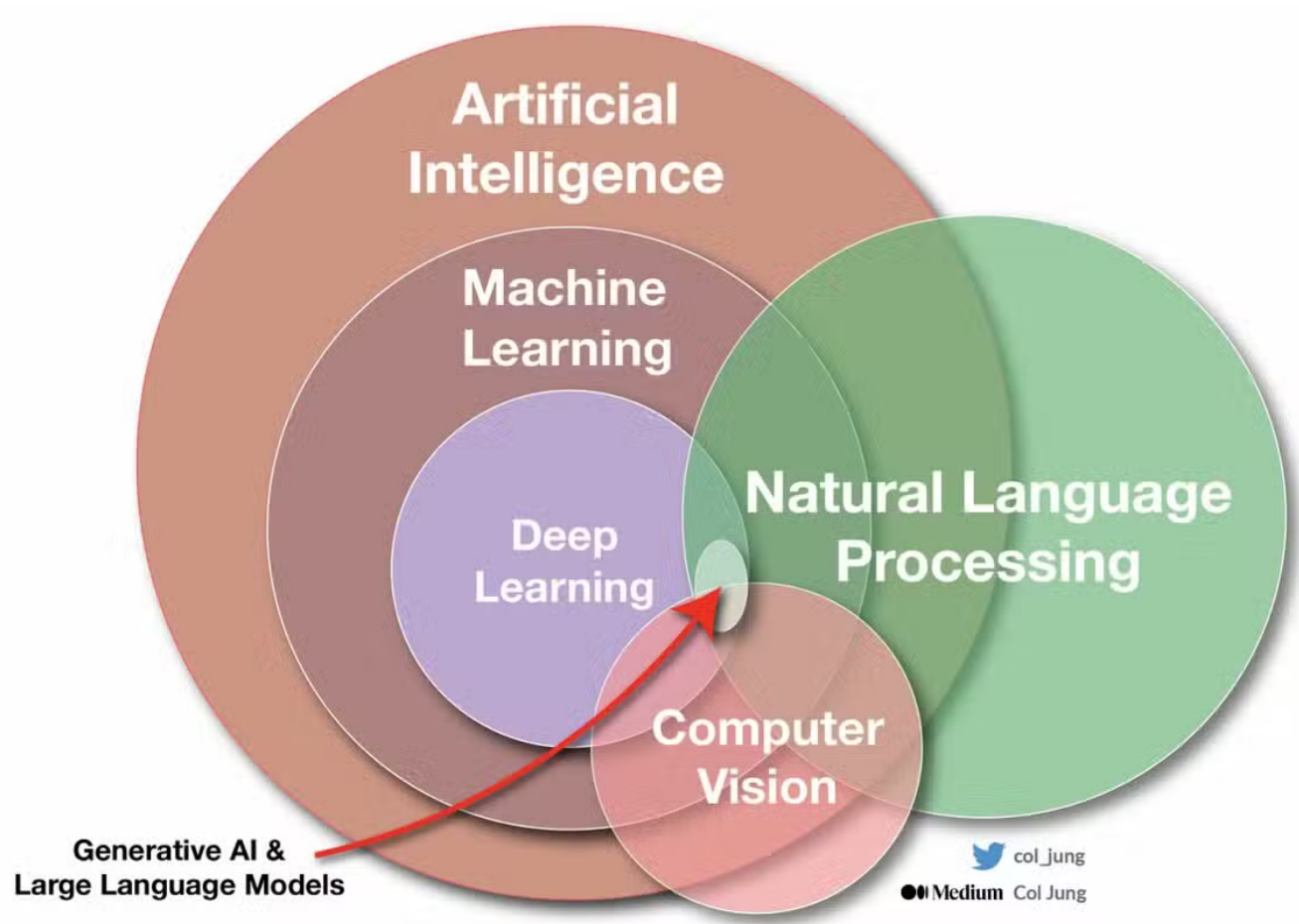
1. Comprendre les applications du TAL.
2. Savoir différencier les étapes de prétraitement de données textuelles et l'intérêt de chaque étape.
3. Lire et valider la structure JSON du pipeline HTR avec jsonschema
4. Quantifier l'impact des lignes `needs_review` et choisir une stratégie de gestion
5. Produire un EDA complet du corpus : distribution, longueurs, abréviations
6. Fixer le split train/val/test en cohérence avec le Volet 1 et calculer le SHA-256
7. Implémenter un normaliseur par règles (Unicode, u/v, i/j, tilde, table d'abréviations)
8. Appliquer une correction guidée par la confiance (utilisation des `char_confidences` et `candidates`)

PARTIE 1 : CONCEPTS FONDAMENTAUX DU TAL ET DU PRÉ-TRAITEMENT

1. INTRODUCTION ET RAPPELS SUR LE TRAITEMENT AUTOMATIQUE DU LANGAGE

QU'EST-CE QUE LE TAL ?

QU'EST-CE QUE LE TAL ?



QU'EST-CE QUE LE TAL ?

- **Définition** : Le TAL (Traitement automatique du langage, NLP) est un domaine multidisciplinaire comprenant la **linguistique informatique** et **l'intelligence artificielle**, qui permet aux machines de comprendre, analyser et générer du langage humain.

QU'EST-CE QUE LE TAL ?

- **Définition** : Le TAL (Traitement automatique du langage, NLP) est un domaine multidisciplinaire comprenant la **linguistique informatique** et **l'intelligence artificielle**, qui permet aux machines de comprendre, analyser et générer du langage humain.
- **Origines**:
 - La *théorie de l'information* de **Shannon** (1940)
 - Les *chaînes* de **Markov** (1950)
 - La *grammaire générative transformationnelle* de **Chomsky** (1950)
 - **Turing** (1950)

2. PRÉ-TRAITEMENT EN NLP: DU TEXTE BRUT À LA DONNÉE EXPLOITABLE

2.1 POURQUOI LE PRÉ-TRAITEMENT TEXTUEL EST DIFFÉRENT DU PRÉ-TRAITEMENT NUMÉRIQUE

- En données numériques : bruit quantitatif (NaN, valeurs aberrantes).
- En données textuelles : bruit symbolique et infini.
- Exemples dans les manuscrits médiévaux :
 - Variations orthographiques (roi, roy, roys, rex)
 - Caractères invisibles (espace de largeur nulle U+200B)
 - Ponctuation irrégulière (signes de coupure médiévaux, tildes)

Conséquence : il faut **normaliser** l'écriture, pas seulement nettoyer.

2.2 TROUVEZ L'ERREUR – LES PIÈGES D'UNICODE

Deux chaînes visuellement identiques peuvent avoir des codages différents.

2.2 TROUVEZ L'ERREUR – LES PIÈGES D'UNICODE

Deux chaînes visuellement identiques peuvent avoir des codages différents.

- "On se retrouve aujourd'hui ?" et "On se retrouve aujourd'hui ?"

2.2 TROUVEZ L'ERREUR – LES PIÈGES D'UNICODE

Deux chaînes visuellement identiques peuvent avoir des codages différents.

- "On se retrouve aujourd'hui ?" et "On se retrouve aujourd'hui ?"
- Le premier utilise l'apostrophe U+0027, le second la guillemet apostrophe courbe U+2019.

2.2 TROUVEZ L'ERREUR – LES PIÈGES D'UNICODE

Deux chaînes visuellement identiques peuvent avoir des codages différents.

- "On se retrouve aujourd'hui ?" et "On se retrouve aujourd'hui ?"
- Le premier utilise l'apostrophe U+0027, le second la guillemet apostrophe courbe U+2019.
- "Tu veux du gâteau ?" / "Tu veux du gâteau ?"

2.2 TROUVEZ L'ERREUR – LES PIÈGES D'UNICODE

Deux chaînes visuellement identiques peuvent avoir des codages différents.

- "On se retrouve aujourd'hui ?" et "On se retrouve aujourd'hui ?"
- Le premier utilise l'apostrophe U+0027, le second la guillemet apostrophe courbe U+2019.
- "Tu veux du gâteau ?" / "Tu veux du gâteau ?"
- Le g peut être la lettre latine U+0067 ou la lettre arménienne U+0581.

2.2 TROUVEZ L'ERREUR – LES PIÈGES D'UNICODE

Deux chaînes visuellement identiques peuvent avoir des codages différents.

- "On se retrouve aujourd'hui ?" et "On se retrouve aujourd'hui ?"
- Le premier utilise l'apostrophe U+0027, le second la guillemet apostrophe courbe U+2019.
- "Tu veux du gâteau ?" / "Tu veux du gâteau ?"
- Le g peut être la lettre latine U+0067 ou la lettre arménienne U+0581.
- J'étais en retard / J'étais en retard / J'étais en retard.

2.2 TROUVEZ L'ERREUR – LES PIÈGES D'UNICODE

Deux chaînes visuellement identiques peuvent avoir des codages différents.

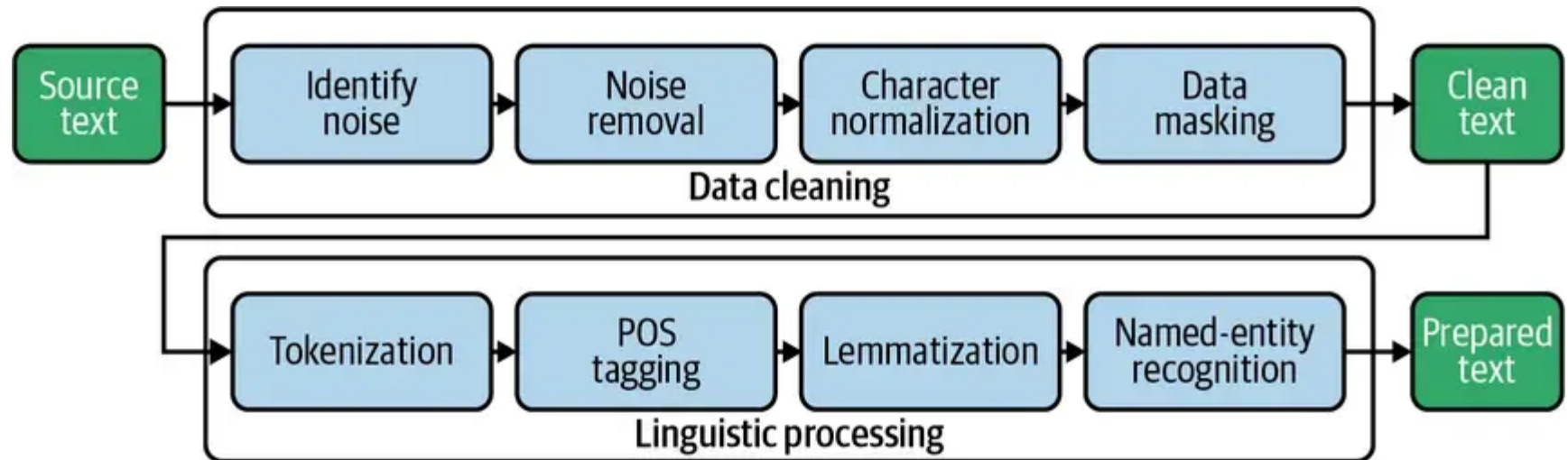
- "On se retrouve aujourd'hui ?" et "On se retrouve aujourd'hui ?"
- Le premier utilise l'apostrophe U+0027, le second la guillemet apostrophe courbe U+2019.
- "Tu veux du gâteau ?" / "Tu veux du gâteau ?"
- Le g peut être la lettre latine U+0067 ou la lettre arménienne U+0581.
- J'étais en retard / J'étais en retard / J'étais en retard.
- Le a peut être latin (U+0061), cyrillique (U+0430) ou alpha latin (U+0251).

2.2 TROUVEZ L'ERREUR – LES PIÈGES D'UNICODE

Deux chaînes visuellement identiques peuvent avoir des codages différents.

- "On se retrouve aujourd'hui ?" et "On se retrouve aujourd'hui ?"
- Le premier utilise l'apostrophe U+0027, le second la guillemet apostrophe courbe U+2019.
- "Tu veux du gâteau ?" / "Tu veux du gâteau ?"
- Le g peut être la lettre latine U+0067 ou la lettre arménienne U+0581.
- J'étais en retard / J'étais en retard / J'étais en retard.
- Le a peut être latin (U+0061), cyrillique (U+0430) ou alpha latin (U+0251).
- références:
 - <https://www.amp-what.com/unicode/search/a>
 - <https://unicodeplus.com/>
 - <https://www.babelstone.co.uk/Unicode/whatisit.html>

3. LES ÉTAPES CLÉS DU PRÉ-TRAITEMENT TEXTUEL



3.1 LA NORMALISATION UNICODE

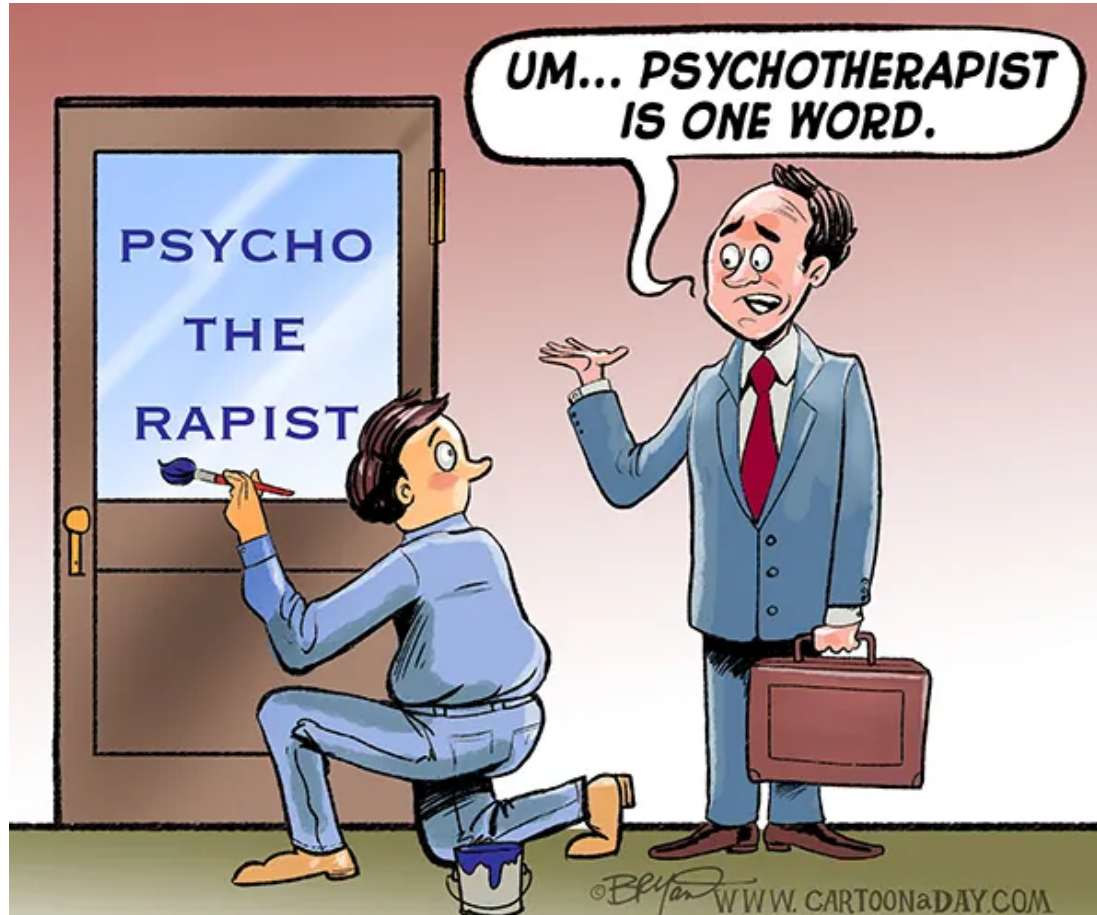
- Deux flottants 0.1 et 0.1000000 deviennent égaux après arrondi.
- En Unicode, un caractère peut être composé (é) ou décomposé (e+accent).
- Forme NFC = « arrondi » de la chaîne.

```
import unicodedata
texte = "cœur" # peut être composé de c + o + e + accent, ou de c + o + e +
ligature ?
texte_norm = unicodedata.normalize('NFC', texte)
```

3.2 LA MISE EN MINUSCULES : L'ÉQUIVALENT DE LA SUPPRESSION DU SIGNE

- Évite que "Paris" et "paris" soient deux mots distincts.
- Sauf pour la reconnaissance d'entités nommées où la casse peut être informative.
- Dans les manuscrits médiévaux, les majuscules sont rares et inconsistantes → on peut tout passer en minuscules.

3.3 LA TOKENISATION : LE DÉCOUPAGE EN UNITÉS



Trois niveaux :

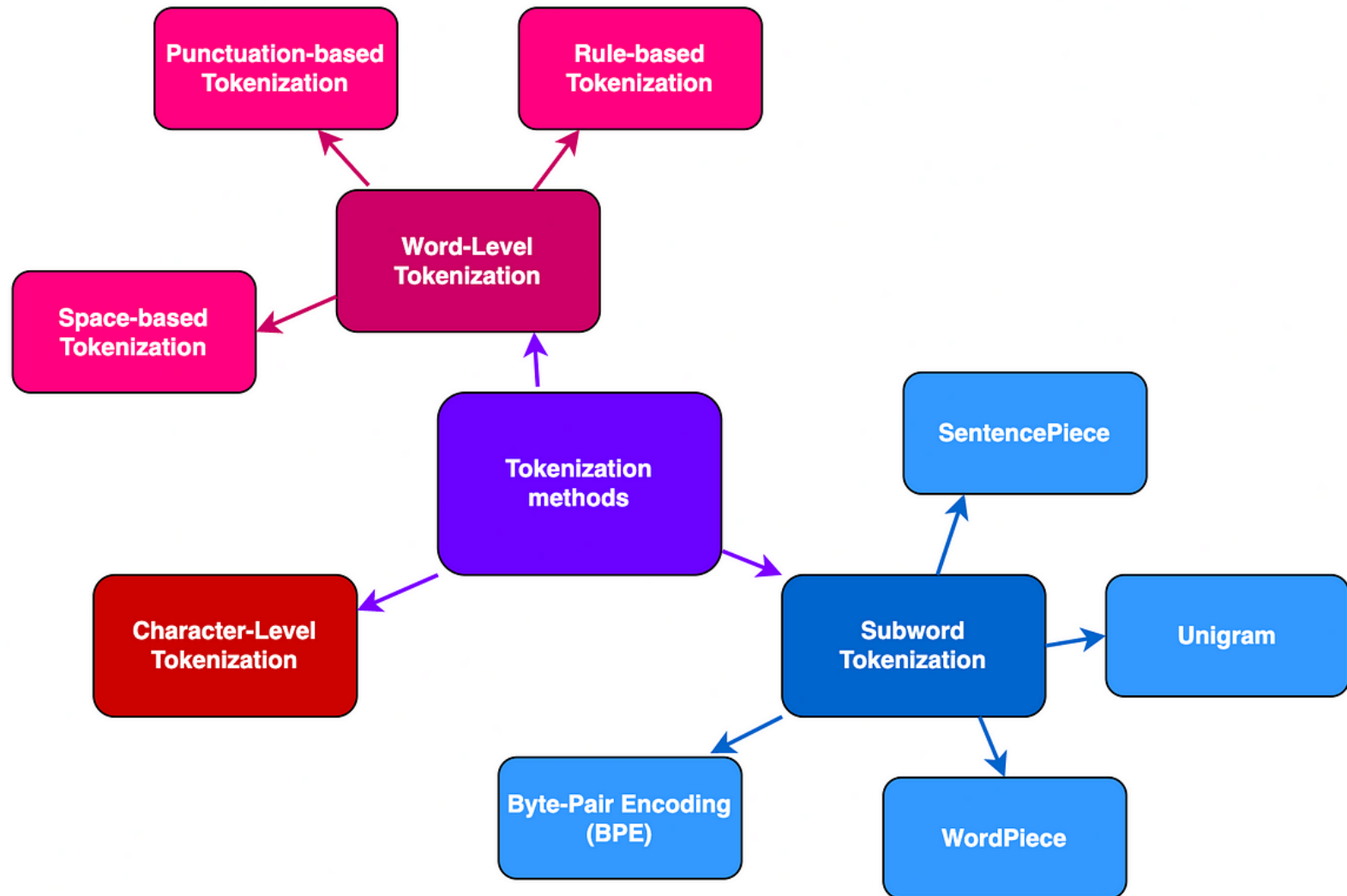
Niveau	Principe	Avantage	Inconvénient
Mots	découpage par espaces/ ponctuation	simple, intuitif	mot hors vocabulaire (OOV) → perte d'information ([UNK])
Sous-mots (BPE, WordPiece)	découpage en fragments fréquents	pas de [UNK], mots rares décomposables	allonge les séquences
Caractères	chaque caractère est un token	0% OOV, très robuste	séquences très longues (3 à 5× plus de tokens)

Problème spécifique au vieux/moyen français Phrase: "Et li cuens prist la dame" Tokenisation CamemBERT :
['_Et', '_li', '_cu', 'ens', '_pr', 'ist', '_la', '_dame']

- cuens coupé en cu + ens → perte de sens sémantique.

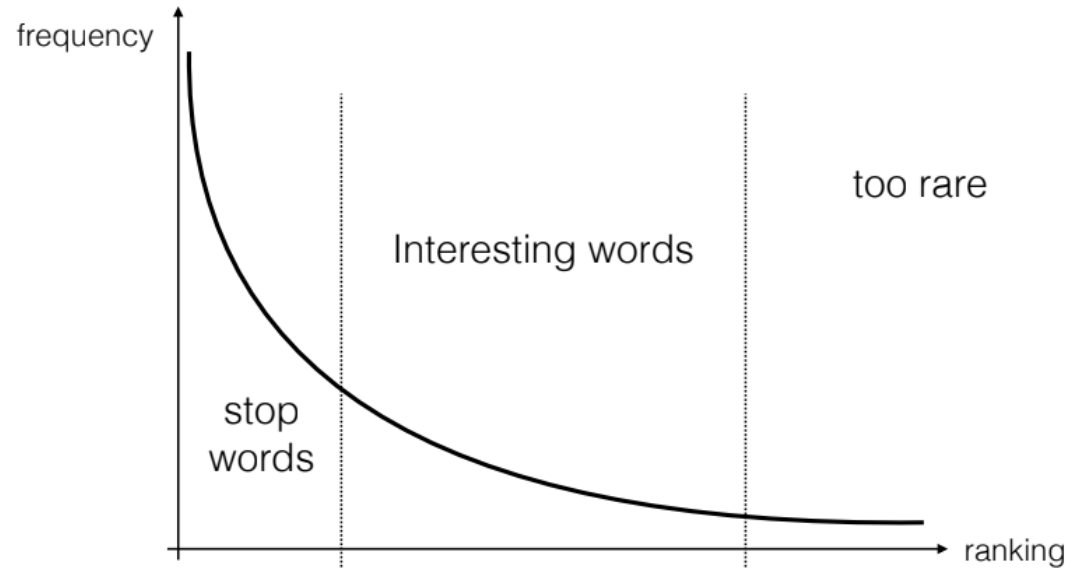
Taux de sous-tokenisation : 45-60% des mots découpés en >1 token.

Tokens [UNK] : 3-5%.



3.4 LA SUPPRESSION DES STOPWORDS : RETIRER CONSTANTES

Zipf's Law



- Analogie : colonne constante en données numériques.
- Stopwords = mots outils très fréquents (le, la, de, et, un).
- À retirer pour certaines tâches (classification de thèmes) mais conservés pour l'attribution d'auteur ou le style.

3.5 LA LEMMATISATION ET LA RACINISATION (STEMMING)

	Racinisation	Lemmatisation
Méthode	supprime suffixes	dictionnaire + analyse grammaticale
Exemple	chantaient → chant	chantaient → chanter
Avantage	rapide	conserve le sens
Inconvénient	perte de sens, mots confondus	plus lent, ressources nécessaires

Pour le projet : lemmatisation préférable (variantes cuens, comte, cunte → comte), mais nécessite un dictionnaire adapté (ex. DMF).

PARTIE 2 : TRAITEMENT DU CORPUS HTR (COMPUTER VISION)

1. STRUCTURE DU DATA CONTRACT HTR

1.1 QU'EST-CE QU'UN DATA CONTRACT ?

- Accord formel sur la structure, le format, les conventions et les garanties des données échangées.
- Équivalent d'une spécification d'API pour les pipelines de données.
- Ici : fichier JSON produit par le pipeline HTR.

1.2 STRUCTURE DU JSON

La structure minimale attendue du data contract est la suivante :

```
{
  "document_id": "gallica_bnf_fr_12483_f003",
  "metadata": {
    "source": "Gallica",
    "century_estimate": "XIV",
    "document_type": "roman",
    "scan_quality": "good",
    "sha256": "a3f2b1..."
  },
  "pages": [
    {
      "page_id": "f003r",
      "image_path": "f003r.tif",
      "lines": [
        {
          "line_id": "f003r_l001",
          "text": "Et li cuens Artus prist la d~e par la main",
          "confidence": 0.87,
          "char_confidences": [0.99, 0.99, 0.88, 0.72, 0.43, ...],
          "candidates": {"position": 4, "options": ["a", "e", "o"]},
          "needs_review": false,
          "polygon": [[120, 45], [892, 45], [892, 68], [120, 68]],
          "reading_order": 1
        }
      ]
    }
  ]
}
```

1.3 CHAMPS CRITIQUES POUR LE MODULE NLP

- **text** : transcription semi-diplomatique (abréviations non résolues conservées).
- **confidence** : score global de la ligne (0–1). $< 0.7 \rightarrow \text{needs_review}$.
- **char_confidences** : liste par caractère, de même longueur que text.
- **candidates** : pour les positions ambiguës, propositions alternatives.
- **needs_review** : booléen. À ne pas ignorer, mais à traiter avec précaution.
- **polygon** : ancrage spatial pour lier entités nommées et image (Jour 4).

2. VALIDATION DU SCHÉMA AVEC jsonschema

- Risques : champ confidence absent, longueur incohérente de char_confidences, needs_review de type string.
- Sans validation, ces incohérences corrompent silencieusement le pipeline.

```
import jsonschema
with open("data_contract_schema.json") as f:
    schema = json.load(f)
jsonschema.validate(instance=htr_data, schema=schema)
```

3. ANALYSE EXPLORATOIRE DU CORPUS (EDA)

3.1 DISTRIBUTIONS À CALCULER EN PRIORITÉ

- **Distribution de confiance** : bimodale si corpus de qualité (pic 0.85-0.95 + pic bas).
- **Taux needs_review par type de document** : chartes > romans (écriture plus standardisée).
- **Longueur des lignes** : médiane 45-65 caractères. Lignes très courtes (<10) = rubriques ou artefacts.
- **Taux d'abréviations résiduelles** : compter les caractères abrégatifs (~, p, q, lettres exposant).

3.2 INTERPRÉTATION DES RÉSULTATS TYPIQUES

Sur un corpus CREMMA moyen-français typique :

- Taux `needs_review` : 15–30% (variable selon la qualité du scan)
- Confiance moyenne : 0.78–0.88
- Abréviations résiduelles : 0.5–3 par ligne selon le type de document (chartes > romans)
- Longueur médiane des lignes : 45–65 caractères

Si votre taux `needs_review` dépasse 40%, le corpus est très difficile : soit la qualité des scans est mauvaise, soit le modèle HTR est sous-performant.

4. STRATÉGIE DE FILTRAGE ET GESTION DES LIGNES

needs_review

- Filtrage **non binaire** mais graduel (« Human-in-the-Loop »).

4.1 DÉFINITION DES SEUILS DE CONFIANCE

Appliquez une logique à trois niveaux. Ces seuils peuvent être ajustés selon le type de champ (nom propre, date, corps de texte) :

Niveau	Confiance	Action
Validation automatique	> 0.90–0.95	Ingestion directe dans le pipeline NLP. Le risque d'erreur est jugé acceptable pour un traitement de masse.
File de révision	0.60–0.90 ou needs_review = true	Extraction dans un fichier séparé (CSV, interface web) pour correction humaine.
Rejet ou traitement manuel total	< 0.60	Exclusion du flux automatique. Signalement pour retranscription manuelle complète ou vérification de l'image source.

Astuce : Dans la file de révision, triez les lignes par score de confiance **croissant** pour que les correcteurs traitent d'abord les cas les plus ambigus, là où l'effort humain est le plus nécessaire.

peut être fatal.

- **Détection d'anomalies** : si écart-type des `char_confidences` > 0.2 , forcer `needs_review`.
- **Post-correction contextuelle** : utiliser CamemBERT (MLM) pour désambiguïser avant revue humaine (détails Jour 2).

notes:

Une ligne avec confiance moyenne 0.85 mais un caractère critique à 0.40 doit être révisée.

notes:

Ne vous fiez **pas uniquement** au score moyen de la ligne. Une ligne peut avoir une confiance globale de 0,85 mais contenir un nom propre critique dont une lettre n'a qu'un score de 0,40 – la différence entre « *Louis* » et « *Louise* » peut être fatale pour une tâche d'extraction d'entités.

- **Détection d'anomalies** : Si la variance des scores caractères dans une ligne est élevée (par exemple écart-type $> 0,2$), cela indique une incohérence interne. Forcez alors le flag `needs_review`, même si la moyenne est bonne.
- **Post-correction contextuelle** : Pour les caractères à faible confiance mais avec des candidats clairs, vous pouvez utiliser un modèle de langage (NLP) pour

désambiguïser automatiquement **avant** la revue humaine. Par exemple, choisir « *a* » plutôt que « *o* » si le contexte exige un verbe. Cette approche sera détaillée au Jour 2.

4.3 WORKFLOW DE CORRECTION

Un workflow typique pour les lignes `needs_review` se décompose en trois étapes :

1. **Extraction** : Parser le JSON pour isoler les lignes `needs_review` et celles dont la confiance tombe dans la zone de révision (0,60–0,90).
2. **Présentation** : Afficher l'image (découpée via les `polygons`) à gauche et le texte éditable à droite, avec les caractères à faible confiance surlignés en rouge. Une interface simple peut être un fichier CSV avec colonnes : `line_id`, `image_crop_path`, `proposed_text` et `corrected_text`.
3. **Réinjection** : Une fois corrigées, ces lignes doivent :
 - Mettre à jour le JSON source (champ `text` et `needs_review` repassé à `false`),
 - Ou alimenter un jeu de **Vérité Terrain** (Ground Truth) pour réentraîner le modèle HTR et améliorer ses futures prédictions.

Bonnes pratiques :

- Conservez un **historique des corrections** (avant/après) pour mesurer l'amélioration du modèle HTR au fil du temps.
- Si le volume de lignes à réviser est trop important ($> 30\%$ du corpus), revoyez d'abord les seuils ou la qualité du pipeline HTR.
- Documentez les décisions de seuillage : dans votre rapport, justifiez pourquoi vous avez choisi 0,90 pour la validation automatique plutôt que 0,95.

5. CONSTRUCTION DU SPLIT TRAIN/VAL/TEST

5.1 PRINCIPE DE STRATIFICATION

Le split doit être **stratifié** selon deux dimensions simultanément :

- Le **siècle** (XIIIe, XIVe, XVe) : pour que chaque partition contienne des textes de toutes les périodes
- Le **type de document** (charte, roman, registre) : pour que le test set reflète la diversité du corpus

Un test set constitué uniquement de chartes du XIVe siècle ne permettra pas d'évaluer la généralisation du modèle.

5.2 LA RÈGLE ABSOLUE : SCELLER LE TEST SET

Le test set doit être **séparé avant tout développement**, et son hachage SHA-256 doit être calculé et consigné immédiatement

```
import hashlib
with open("test_set.json", "rb") as f:
    print(hashlib.sha256(f.read()).hexdigest())
```

Règle absolue : une fois le test set scellé, il ne doit plus être regardé jusqu'au rendu final. Toutes les décisions d'architecture, d'hyperparamètres et d'ensembles de données d'entraînement sont prises en observant uniquement la performance sur le jeu de **validation**.

6. NORMALISATION ORTHOGRAPHIQUE : MODULE DE RÈGLES

6.1 ORGANISATION DU CODE

- Classe Python indépendante, documentée, avec tests automatisés.
- Possibilité de désactiver chaque règle pour mesurer son impact.

6.2 RÈGLES PRINCIPALES

- **Normalisation Unicode** : `unicodedata.normalize('NFC', text)`
- **Substitution u/v** : $u \rightarrow v$ en début de mot ou devant voyelle, $v \rightarrow u$ ailleurs.
- **Substitution i/j** : $i \rightarrow j$ devant voyelle.
- **Résolution du tilde nasal** : $\sim$ après a, e, o $\rightarrow +n$; $\sim$ après o en fin de mot $\rightarrow +m$.
- **Abréviations courantes** : table JSON (ex: "q~" : "que", "p" : "par", "d~e" : "dame").

Ces règles sont inspirées des pratiques de la philologie médiévale. La table d'abréviations peut être enrichie à partir du DMF.

6.3 MESURER L'IMPACT DES RÈGLES

- Pour chaque règle, calculer le CER **avant** et **après** application sur 200 lignes de référence.
- Tableau d'ablation.

```
cer_before = cer(reference, text_before)
cer_after = cer(reference, text_after)
print(f"Gain: {cer_before - cer_after:.2f}%")
```

7. CORRECTION CONTEXTUELLE GUIDÉE PAR CONFIANCE

7.1 LE PRINCIPE

Le pipeline HTR fournit, pour chaque ligne, les `char_confidences` et les `candidates` pour les positions ambiguës. Plutôt que de traiter chaque caractère indépendamment, on utilise un modèle de langue (CamemBERT en mode MLM) pour arbitrer entre les candidats proposés.

1. **Identification** : Parcourir les `char_confidences` de chaque ligne. Repérer les positions dont la confiance est inférieure à un seuil (ex. 0,7) et qui possèdent une entrée `candidates`.
2. **Arbitrage par modèle de langue** : Pour chaque position ambiguë, générer les variantes du texte en remplaçant le caractère par chaque option proposée. Utiliser un modèle de type CamemBERT en mode MLM pour évaluer la vraisemblance de chaque variante (masquer le mot complet contenant la position).
3. **Application** : Choisir l'option avec le score MLM le plus élevé et modifier le texte de la ligne en conséquence. Conserver une trace des corrections dans un fichier de log.
4. **Réinjection (optionnelle)** : Les lignes ainsi corrigées peuvent servir à mettre à jour le data contract (passage de `needs_review` à `false`) ou à alimenter un jeu de vérité terrain pour réentraîner le modèle HTR.

FIN DU J1

**PROCHAINE SÉANCE : FINE-TUNING T5-LORA POUR LA
NORMALISATION DES CAS RÉSIDUELS.**